

IES Isidra de Guzmán
Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Multiplataforma

SimuTrade

Simulador educativo de bolsa y criptomonedas

Memoria del Proyecto Intermodular

Autores:

Guillermo Sierra Castejón

Iván Bertolo García

Ximena Meyzen Calderón

Tutora: Ruth Lospitao Ruiz

Curso académico 2025/2026

Fecha de presentación: 27/05/2026

Agradecimientos

Los autores desean expresar su agradecimiento a la tutora Ruth Lospitao Ruiz por su orientación y apoyo durante el desarrollo de este proyecto. Asimismo, agradecen a todos los compañeros y profesores del ciclo los conocimientos, consejos y ayuda aportados, que han sido fundamentales para la realización de este trabajo y para llegar hasta este punto.

Resumen

SimuTrade es una aplicación móvil Android desarrollada en Kotlin con Jetpack Compose cuyo objetivo principal es permitir que cualquier persona aprenda cómo funcionan los mercados financieros de forma segura, sin utilizar dinero real. Se ha diseñado especialmente para jóvenes sin experiencia previa en inversión, eliminando la principal barrera de aprendizaje: el miedo a perder dinero.

El usuario comienza con un saldo virtual de 100 euros y puede comprar y vender acciones de empresas como Apple, Microsoft o Tesla, así como criptomonedas como Bitcoin o Ethereum, con precios obtenidos en tiempo real de las APIs CoinGecko y Finnhub. La aplicación incorpora un sistema de seis rangos (Principiante, Bronce, Plata, Oro, Platino y Diamante) basado exclusivamente en el beneficio obtenido por trading, una tabla de clasificación global, un sistema de retos diarios con racha y recompensas, y un sistema social completo con amigos y ligas privadas de competición, incluyendo un chat en tiempo real entre los miembros de cada liga.

El backend está implementado íntegramente sobre Firebase (Firebase, 2024), utilizando Firebase Authentication para la gestión de usuarios y Cloud Firestore como base de datos NoSQL en tiempo real. La arquitectura sigue el patrón MVVM con repositorios separados por dominio.

Palabras clave: Android, Kotlin, Jetpack Compose, Firebase, inversión educativa, MVVM.

Abstract

SimuTrade is an Android mobile application developed in Kotlin with Jetpack Compose whose main objective is to allow anyone to learn how financial markets work safely, without using real money. It has been especially designed for young people with no prior investment experience, removing the main learning barrier: the fear of losing money.

The user starts with a virtual balance of 100 euros and can buy and sell shares from companies such as Apple, Microsoft or Tesla, as well as cryptocurrencies such as Bitcoin or Ethereum, with prices obtained in real time from the CoinGecko and Finnhub APIs. The application incorporates a ranking system based exclusively on trading profit, a global leaderboard, a daily challenges system with streaks and rewards, and a complete social system with friends and private competition leagues, including a real-time chat between the members of each league.

The backend is fully implemented on Firebase, using Firebase Authentication for user management and Cloud Firestore as a real-time NoSQL database. The architecture follows the MVVM pattern with domain-separated repositories.

Keywords: Android, Kotlin, Jetpack Compose, Firebase, investment education, MVVM.

Índice General

Agradecimientos.....	2
Resumen	3
Abstract	3
Índice General	5
Índice de Figuras y Tablas	7
Índice de Figuras	7
Índice de Tablas	7
1. Introducción y Objetivos del Proyecto	8
1.1 Motivación.....	8
1.2 Objetivos del Proyecto	8
2. Análisis del Problema y Justificación de la Solución	10
2.1 Análisis del Problema	10
2.2 Justificación de la Solución.....	10
2.3 Requisitos Funcionales.....	11
2.4 Requisitos No Funcionales	12
3. Diseño del Sistema	13
3.1 Arquitectura de la Aplicación	13
3.2 Modelo de Datos — Cloud Firestore.....	15
3.3 Diagrama de Casos de Uso.....	16
3.4 Diagrama de Clases	17
4. Desarrollo del Proyecto	19
4.1 Tecnologías y Herramientas Utilizadas.....	19
4.2 Descripción de la Arquitectura	20
4.3 Capturas de Pantalla — Descripción de Funcionalidades	22
4.3.1 Autenticación.....	22
4.3.2 Panel de Control (Dashboard).....	24
4.3.3 Mercado	25
4.3.4 Operaciones de Trading.....	26
4.3.5 Ranking Global.....	27
4.3.6 Retos Diarios.....	28
4.3.7 Sistema de Amigos	29
4.3.8 Sistema de Ligas y Chat	31
4.3.9 Perfil de Usuario.....	34
5. Pruebas Realizadas y Validación	35

5.1 Pruebas de Autenticación	35
5.2 Pruebas de Trading	36
5.3 Pruebas del Sistema de Retos	36
5.4 Pruebas del Sistema Social y Chat.....	37
6. Planificación del Proyecto	38
7. Conclusiones y Mejoras Futuras	39
7.1 Objetivos Alcanzados	39
7.2 Dificultades Encontradas	39
7.3 Posibles Mejoras Futuras	40
Bibliografía	41
Anexos	42
Anexo I — Manual de Usuario	42
A.1.1 Registro e inicio de sesión	42
A.1.2 Operar en el mercado	42
A.1.3 Sistema de retos	42
A.1.4 Añadir amigos y crear ligas.....	42
Anexo II — Manual de Instalación y Ejecución	43
A.2.1 Requisitos previos.....	43
A.2.2 Clonado e importación del proyecto.....	43
A.2.3 Configuración de Firebase.....	43
A.2.4 Configuración de la API de Finnhub	43
A.2.5 Ejecución	44
A.2.6 Instalación mediante APK.....	44
Anexo III — Declaración de Originalidad	44

Índice de Figuras y Tablas

Índice de Figuras

- Figura 1. Estructura de paquetes del proyecto en Android Studio.
- Figura 2. Esquema del modelo de datos en Firestore.
- Figura 3. Pantalla de inicio de sesión (LoginScreen).
- Figura 4. Pantalla de registro (RegistroScreen).
- Figura 5. Panel de control del usuario (DashboardScreen).
- Figura 6. Pantalla de mercado con lista de activos (MercadoScreen).
- Figura 7. Pantalla de operaciones de compra/venta (OperacionesScreen).
- Figura 8. Pantalla de ranking global (RankingScreen).
- Figura 9. Pantalla de retos diarios (RetosScreen).
- Figura 10. Pantalla de amigos — lista y solicitudes (AmigosScreen).
- Figura 11. Pantalla de amigos — búsqueda por código.
- Figura 12. Pantalla de ligas — lista de ligas activas.
- Figura 13. Pantalla de ligas — ranking y chat de una liga.
- Figura 14. Diálogo de perfil de usuario (PerfilScreen).
- Figura 15. Documento del usuario en Firebase Console tras el registro.

Índice de Tablas

- Tabla 1. Arquitectura MVVM — capas y responsabilidades.
- Tabla 2. Requisitos funcionales del sistema.
- Tabla 3. Requisitos no funcionales del sistema.
- Tabla 4. Caso de uso CU-01: Comprar activo.
- Tabla 5. Caso de uso CU-02: Completar reto diario.
- Tabla 6. Caso de uso CU-03: Crear liga.
- Tabla 7. Diagrama de clases simplificado.
- Tabla 8. Tecnologías y herramientas utilizadas en el proyecto.
- Tabla 9. Estructura de paquetes del proyecto.
- Tabla 10. Resultados de las pruebas — registro e inicio de sesión.
- Tabla 11. Resultados de las pruebas — compra y venta de activos.
- Tabla 12. Resultados de las pruebas — sistema de retos.
- Tabla 13. Resultados de las pruebas — sistema social y chat.
- Tabla 14. Planificación del proyecto por fases.

1. Introducción y Objetivos del Proyecto

1.1 Motivación

La idea de SimuTrade surge de la necesidad de acercar el mundo de las finanzas y la inversión a personas jóvenes que nunca han tenido contacto con la bolsa de valores. Generalmente, el desconocimiento financiero es uno de los principales problemas de la sociedad actual, y el miedo a perder dinero real es la barrera más frecuente que impide a los jóvenes aprender a invertir.

Se ha observado que la mayoría de las aplicaciones de simulación de bolsa existentes presentan interfaces complejas poco adecuadas para usuarios sin experiencia, carecen de elementos de gamificación que mantengan la motivación del usuario o no disponen de funcionalidades sociales que fomenten la competición sana entre usuarios.

SimuTrade pretende resolver estos problemas ofreciendo una experiencia de aprendizaje financiero gamificada, intuitiva y social, donde el usuario aprende mientras se divierte y compite con sus amigos en ligas privadas con chat en tiempo real.

1.2 Objetivos del Proyecto

Los objetivos principales establecidos al inicio del proyecto son los siguientes:

- Desarrollar una aplicación Android completa con arquitectura MVVM y Jetpack Compose (Android Developers, 2024) que siga las guías de diseño de Material Design 3.
- Implementar un backend escalable con Firebase Authentication y Cloud Firestore (Firebase, 2024) que gestione usuarios, carteras y transacciones en tiempo real.
- Integrar APIs externas reales (CoinGecko, 2024; Finnhub, 2024) para obtener precios actualizados de acciones y criptomonedas con gestión adecuada de rate limiting.

- Diseñar un sistema de rangos basado exclusivamente en el beneficio obtenido por operaciones de trading, independiente de las recompensas por retos.
- Implementar un sistema de retos diarios con validación real contra datos de Firestore, racha y recompensas en saldo bonus separado.
- Crear un sistema social completo con búsqueda de usuarios por código identificativo generado automáticamente, solicitudes de amistad y ligas privadas con ranking y chat en tiempo real.
- Garantizar la equidad del sistema de rangos y del leaderboard descartando el saldo bonus de retos en los cálculos de beneficio.

2. Análisis del Problema y Justificación de la Solución

2.1 Análisis del Problema

Se ha llevado a cabo un análisis de las principales aplicaciones de simulación de bolsa disponibles en el mercado, identificando las siguientes carencias:

- Interfaces complejas orientadas a inversores profesionales, no a usuarios principiantes.
- Ausencia de elementos de gamificación como retos, rachas o sistemas de recompensa que mantengan la motivación.
- Falta de funcionalidades sociales como rankings entre amigos, ligas privadas o chat entre participantes.
- Uso de datos estáticos o simulados en lugar de precios reales actualizados.

Adicionalmente, se ha identificado que el uso de SharedPreferences para persistencia local, solución habitual en proyectos académicos previos, presenta limitaciones importantes: los datos se pierden al reinstalar la aplicación, no permite la sincronización entre dispositivos y no es adecuada para un sistema multiusuario.

2.2 Justificación de la Solución

Para resolver los problemas identificados se ha optado por las siguientes decisiones de diseño:

- Firebase Authentication y Cloud Firestore como backend (Firebase, 2024), eliminando la necesidad de un servidor propio y garantizando persistencia, sincronización en tiempo real y escalabilidad.
- Material Design 3 con Jetpack Compose para una interfaz moderna, intuitiva y accesible para usuarios sin experiencia previa.
- APIs reales de CoinGecko (2024) y Finnhub (2024) para precios actualizados, con gestión de rate limiting para evitar errores 429.

- Sistema de retos diarios con validación real y saldo bonus independiente del beneficio de trading, garantizando la equidad del ranking.
- Código identificativo generado automáticamente con verificación de unicidad en Firestore para identificar a los usuarios en el sistema social sin exponer datos personales.

2.3 Requisitos Funcionales

Tabla 2. Requisitos funcionales del sistema.

ID	Descripción
RF-01	El sistema permite al usuario registrarse con email/contraseña o con Google Sign-In.
RF-02	El usuario dispone de un saldo virtual inicial de 100 euros para invertir.
RF-03	El sistema muestra la lista de activos disponibles con precios reales actualizados.
RF-04	El usuario puede comprar un activo si dispone de saldo suficiente.
RF-05	El usuario puede vender un activo si dispone de cantidad suficiente en cartera.
RF-06	El sistema registra todas las transacciones realizadas por el usuario.
RF-07	El sistema calcula el beneficio de trading descartando el saldo bonus de retos.
RF-08	El sistema asigna un rango al usuario según su beneficio de trading acumulado.
RF-09	El sistema muestra un leaderboard global ordenado por beneficio de trading.
RF-10	El sistema genera 3 retos diarios aleatorios y los resetea cada medianoche.
RF-11	Al completar un reto, el sistema valida las condiciones consultando Firestore en tiempo real.
RF-12	Las recompensas de retos se acumulan en un saldo bonus separado del saldo de trading.
RF-13	El usuario puede buscar otros usuarios por su código identificativo.
RF-14	El usuario puede enviar y aceptar solicitudes de amistad.
RF-15	El usuario puede crear ligas privadas con nombre personalizado.
RF-16	El usuario puede invitar a sus amigos a sus ligas y aceptar invitaciones recibidas.
RF-17	Cada liga muestra su propio ranking calculado con los datos reales de Firestore de cada miembro.
RF-18	El usuario puede enviar y recibir mensajes dentro de las ligas privadas a las que pertenece, mediante un chat en tiempo real asociado a cada liga.

Nota. Fuente: elaboración propia.

2.4 Requisitos No Funcionales

Tabla 3. Requisitos no funcionales del sistema.

ID	Descripción
RNF-01	La interfaz sigue las guías de Material Design 3 y es compatible con modo oscuro.
RNF-02	La aplicación responde a las acciones del usuario en menos de 1 segundo en condiciones normales de red.
RNF-03	Los datos se persisten en Cloud Firestore y se mantienen entre sesiones y dispositivos.
RNF-04	La aplicación es compatible con dispositivos Android con API mínima 24 (Android 7.0).
RNF-05	El código sigue la arquitectura MVVM con repositorios separados por dominio funcional.
RNF-06	Las peticiones a CoinGecko respetan el rate limit mediante un cooldown de 60 segundos.
RNF-07	El saldo bonus de retos no influye en el cálculo del rango ni en el leaderboard global.
RNF-08	El sistema utiliza batch writes en operaciones que afectan a varios documentos relacionados, especialmente en el sistema social de amigos y ligas, reduciendo el riesgo de inconsistencias en cambios simultáneos.
RNF-09	Los mensajes del chat de liga se almacenan en Cloud Firestore y se muestran de forma actualizada a todos los miembros de la liga mediante addSnapshotListener.

Nota. Fuente: elaboración propia.

3. Diseño del Sistema

3.1 Arquitectura de la Aplicación

SimuTrade sigue el patrón de arquitectura MVVM (Model - View - ViewModel), que es el patrón recomendado por Google para aplicaciones Android con Jetpack Compose (Android Developers, 2024). Este patrón separa claramente las responsabilidades de cada capa, facilitando el testing, el mantenimiento y la escalabilidad del código.

Tabla 1. Arquitectura MVVM — capas y responsabilidades.

Capa	Paquete	Responsabilidad
Model	datos/modelo/	Define las estructuras de datos del dominio: DatosUsuario, Activo, Liga, MensajeChat, Reto, etc.
Repository	datos/repositorio/	Contiene la lógica de negocio y el acceso a Firebase y APIs externas. Separado por dominio funcional.
ViewModel	ui/*/ViewModel	Gestiona el estado de la UI mediante StateFlow. Conecta los repositorios con las vistas.
View	ui/*/Screen	Pantallas declarativas construidas con Jetpack Compose. No contienen lógica de negocio.

Nota. Fuente: elaboración propia.

Como se muestra en la Figura 1, la estructura de paquetes del proyecto refleja directamente esta separación por capas y por dominio funcional.

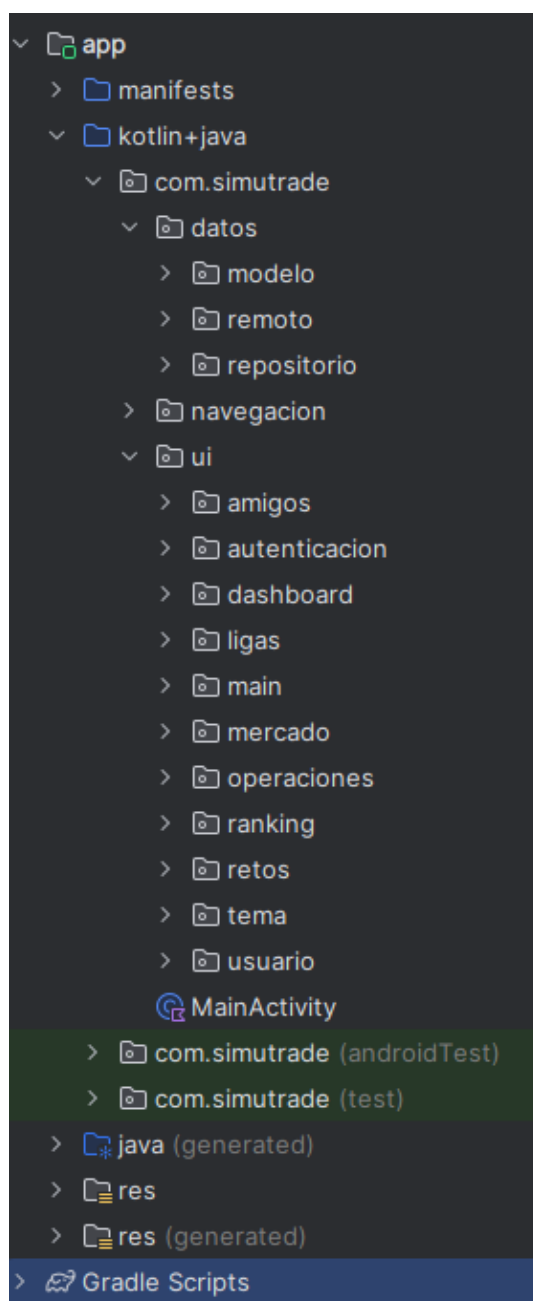


Figura 1. Estructura de paquetes del proyecto en Android Studio. Fuente: captura propia de la aplicación.

3.2 Modelo de Datos — Cloud Firestore

La base de datos está implementada en Cloud Firestore (Firebase, 2024), una base de datos NoSQL orientada a documentos. Se ha optado por este modelo en lugar de una base de datos relacional por su mayor flexibilidad para almacenar estructuras de datos anidadas, su sincronización en tiempo real y su integración nativa con el ecosistema Firebase.

La estructura se organiza en dos colecciones raíz: Usuarios y Ligas. Dentro de cada documento de usuario se encuentran las subcolecciones que almacenan la cartera, las transacciones, los datos de retos, los amigos, las solicitudes pendientes y las invitaciones a ligas recibidas. Dentro de cada documento de liga se almacena la subcolección miembros con el estado de cada participante, y la subcolección mensajes donde se guardan los mensajes del chat. Cada mensaje contiene el identificador del autor, el nombre del usuario, el texto del mensaje y la fecha de envío.

Como se ilustra en la Figura 2, el campo saldo_bonus almacena de forma independiente el dinero obtenido mediante retos, lo que permite calcular el beneficio de trading puro restando dicho bonus antes de determinar el rango del usuario.

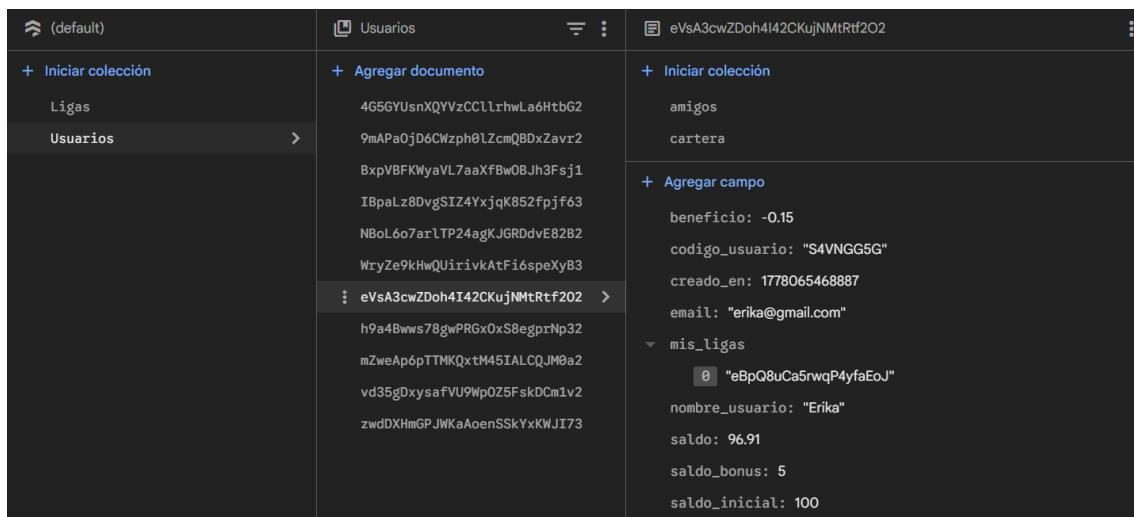


Figura 2. Esquema del modelo de datos en Firestore. Fuente: captura propia de la aplicación.

3.3 Diagrama de Casos de Uso

A continuación, se describen los principales casos de uso del sistema. El actor principal es el Usuario autenticado.

Tabla 4. Caso de uso CU-01: Comprar activo.

Campo	Descripción
Nombre	CU-01: Comprar activo
Actor	Usuario autenticado
Precondición	El usuario dispone de saldo suficiente para la compra.
Flujo principal	1. El usuario accede a la pantalla de Mercado. 2. Selecciona un activo. 3. Introduce la cantidad. 4. Pulsa Comprar. 5. El sistema descuenta el importe del saldo, actualiza la cartera y registra la transacción en Firestore.
Flujo alternativo	Si el saldo es insuficiente, el sistema muestra un mensaje de error y no realiza la operación.
Postcondición	El activo aparece en la cartera del usuario y el beneficio de trading se recalcula.

Nota. Fuente: elaboración propia.

Tabla 5. Caso de uso CU-02: Completar reto diario.

Campo	Descripción
Nombre	CU-02: Completar reto diario
Actor	Usuario autenticado
Precondición	El reto no ha sido completado hoy y el usuario puede cumplir las condiciones.
Flujo principal	1. El usuario accede a la pantalla de Retos. 2. Pulsa Completar en un reto. 3. El sistema valida las condiciones consultando Firestore en tiempo real. 4. Si se cumplen, añade la recompensa al saldo bonus y marca el reto. 5. Si se completan todos los retos del día, incrementa la racha.
Flujo alternativo	Si las condiciones no se cumplen, el sistema muestra un mensaje indicando exactamente qué falta por cumplir.
Postcondición	El saldo bonus aumenta y el reto queda marcado como completado hasta el siguiente reset diario.

Nota. Fuente: elaboración propia.

Tabla 6. Caso de uso CU-03: Crear liga.

Campo	Descripción
Nombre	CU-03: Crear liga
Actor	Usuario autenticado
Precondición	El usuario está autenticado.
Flujo principal	1. El usuario accede a la pantalla de Ligas. 2. Pulsa el botón +. 3. Introduce el nombre de la liga. 4. El sistema crea el documento de la liga en Firestore y añade al creador como miembro aceptado. 5. El usuario puede invitar a sus amigos.
Flujo alternativo	Si el nombre está vacío, el sistema muestra un error de validación y no crea la liga.
Postcondición	La liga aparece en la lista de ligas del usuario con ranking y chat disponibles.

Nota. Fuente: elaboración propia.

3.4 Diagrama de Clases

La Tabla 7 muestra las principales clases del sistema y sus atributos más relevantes. Se han utilizado data classes de Kotlin para todos los modelos de datos, garantizando inmutabilidad y facilitando la comparación de estados en los StateFlow del patrón MVVM (Android Developers, 2024).

Tabla 7. Diagrama de clases simplificado.

Clase	Tipo	Atributos principales
DatosUsuario	data class	idUsuario, nombreUsuario, email, saldo, saldoInicial, saldoBonus, codigoUsuario
Activo	data class	id, simbolo, nombre, tipo, precioActual, cambioPrecio24h, cambioPorcentaje24h
ActivoEnCartera	data class	idActivo, simbolo, cantidad, precioPromedio, precioActual, valorActual, beneficio
Transaccion	data class	id, fecha, tipo, idActivo, simbolo, cantidad, precio, total
Rango	data class	nombre, beneficioMinimo — 6 rangos: Principiante (negativo), Bronce (0 €+), Plata (50 €+), Oro (150 €+), Platino (300 €+), Diamante (500 €+)
EntradaRanking	data class	id, nombreUsuario, beneficio, valorTotal, valorCartera, saldo
DatosRetos	data class	rachaActual, rachaMaxima, retosCompletados, retosDelDia, diaActual
Reto	data class	id, titulo, descripcion, recompensa
Amigo	data class	uid, nombreUsuario, codigoUsuario
SolicitudAmistad	data class	uid, nombreUsuario, codigoUsuario, enviadoEn
Liga	data class	id, nombre, creadoPor, creadoEn, miembros
MiembroLiga	data class	uid, nombreUsuario, codigoUsuario, estado, invitadoPor
InvitacionLiga	data class	ligaid, nombreLiga, invitadoPor, creadoEn
MensajeChat	data class	id, uid, nombreUsuario, texto, enviadoEn
TipoActivo	enum	ACCION, CRIPTO
TipoTransaccion	enum	COMPRA, VENTA
ResultadoOperacion	sealed class	Exito(mensaje, usuario), Error(mensaje)
ResultadoAutenticacion	sealed class	Exito(usuario), Error(mensaje)

Nota. Fuente: elaboración propia.

4. Desarrollo del Proyecto

4.1 Tecnologías y Herramientas Utilizadas

A continuación, se describen las tecnologías y herramientas empleadas en el desarrollo del proyecto, justificando la elección de cada una de ellas.

Tabla 8. Tecnologías y herramientas utilizadas en el proyecto.

Tecnología	Versión	Uso en el proyecto
Kotlin	2.0.21	Lenguaje principal de desarrollo para Android.
Jetpack Compose	BOM 2025.04.01	Framework declarativo para construir la interfaz gráfica.
Firebase Authentication	BOM 33.12.0	Gestión de usuarios con email/contraseña y Google Sign-In.
Cloud Firestore	BOM 33.12.0	Base de datos NoSQL en tiempo real para persistencia de datos.
CoinGecko API	v3 Free	Precios en tiempo real de criptomonedas.
Finnhub API	v1	Cotizaciones en tiempo real de acciones del mercado americano.
Retrofit	2.11.0	Librería para peticiones HTTP REST a las APIs externas.
OkHttp	4.12.0	Cliente HTTP subyacente de Retrofit con interceptores.
Material Design 3	BOM 2025.04.01	Sistema de diseño para la interfaz de usuario.
Android Studio	Meerkat 2024.3.1	Entorno de desarrollo integrado oficial para Android.
GitHub	—	Control de versiones y trabajo colaborativo.

Nota. Fuente: elaboración propia.

4.2 Descripción de la Arquitectura

La aplicación se organiza en dos grandes módulos: el módulo de datos y el módulo de interfaz de usuario. El módulo de datos contiene los modelos, las interfaces de las APIs remotas y los repositorios. El módulo de interfaz contiene los ViewModels, las pantallas Composable y el tema visual.

La navegación entre pantallas se gestiona mediante Jetpack Navigation Compose, con un grafo de navegación definido en NavGraph.kt que detecta reactivamente el estado de sesión de Firebase mediante un AuthStateListener. Esto garantiza que el usuario sea redirigido automáticamente a la pantalla de login cuando cierra sesión, y a la pantalla principal cuando la sesión está activa.

Tabla 9. Estructura de paquetes del proyecto.

Paquete	Contenido
datos/modelo/	Modelos.kt — todas las clases de datos del dominio de la aplicación.
datos/remoto/	CoinGeckoApi.kt, FinnhubApi.kt — interfaces Retrofit para las APIs externas de precios de mercado.
datos/repositorio/	RepositorioUsuario, RepositorioAutenticacion, RepositorioMercado, RepositorioAmigos, RepositorioLigas.
navegacion/	NavGraph.kt — grafo de navegación con detección reactiva del estado de sesión de Firebase.
ui/autenticacion/	LoginScreen, RegistroScreen, AutenticacionViewModel.
ui/main/	MainScreen, MainViewModel, PerfilScreen — navegación principal y perfil.
ui/dashboard/	DashboardScreen — resumen de cartera, rango y transacciones.
ui/mercado/	MercadoScreen, MercadoViewModel — listado de activos con precios reales.
ui/operaciones/	OperacionesScreen — compra y venta de activos.
ui/ranking/	RankingScreen, RankingViewModel, RankingUtilidades — leaderboard global.
ui/retos/	RetosScreen, RetosViewModel — retos diarios con racha y recompensas.
ui/amigos/	AmigosScreen, AmigosViewModel — sistema social de amigos.
ui/ligas/	LigasScreen, LigasViewModel — ligas privadas con ranking y chat en tiempo real.
ui/usuario/	UsuarioViewModel — lógica de trading, cálculo de beneficios y rangos.
ui/tema/	Tema.kt, Tipografia.kt — colores, tipografías y extensiones de ColorScheme.
MainActivity.kt	Punto de entrada de la aplicación. Configura el tema visual y lanza el NavGraph.

Nota. Fuente: elaboración propia.

4.3 Capturas de Pantalla — Descripción de Funcionalidades

4.3.1 Autenticación

La aplicación cuenta con dos métodos de autenticación: email y contraseña, gestionado por Firebase Authentication, y Google Sign-In mediante GoogleAuthProvider. Al registrarse, el sistema genera automáticamente un código identificativo de 8 caracteres (letras A-Z y números 0-9) con verificación de unicidad en Firestore, que queda asociado al perfil del usuario y permite identificarlo en el sistema social con el formato #XXXXXXXX.

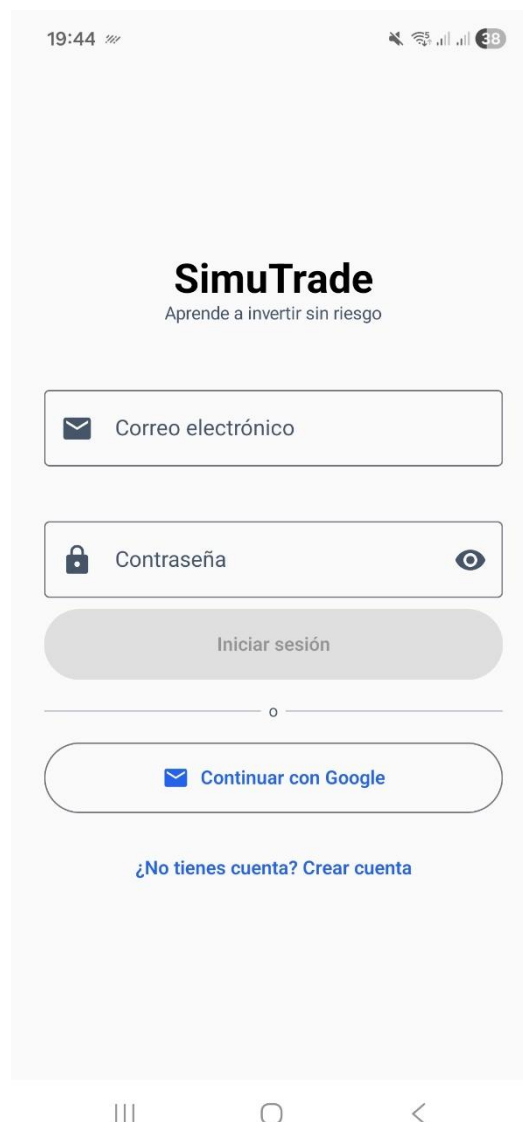


Figura 3. Pantalla de inicio de sesión con email/contraseña y Google Sign-In. Fuente: captura propia de la aplicación.

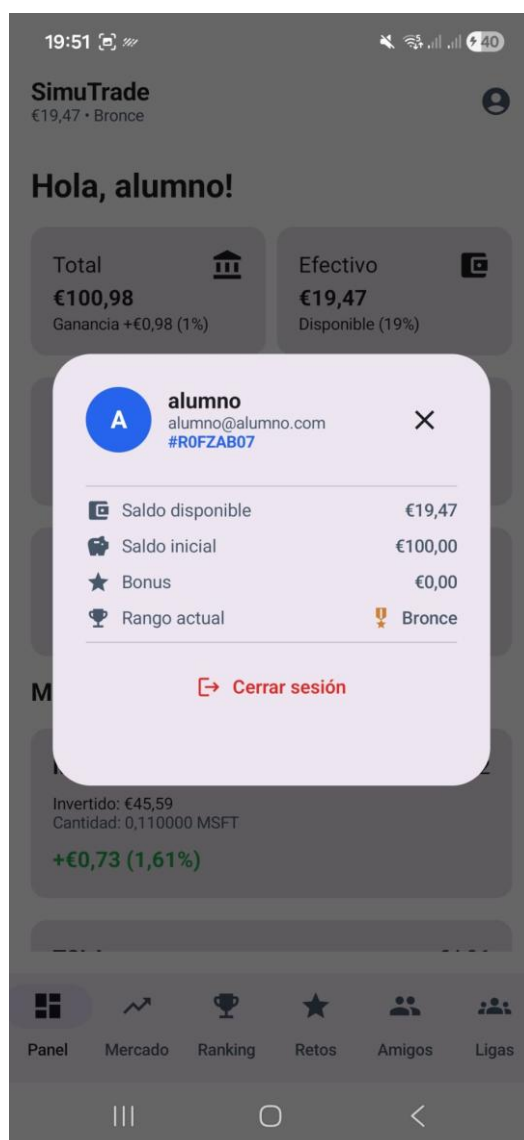


Figura 4. Pantalla de registro con generación automática de código identificativo. Fuente: captura propia de la aplicación.

4.3.2 Panel de Control (Dashboard)

La pantalla principal muestra el resumen completo de la situación financiera del usuario: valor total del portfolio, efectivo disponible, valor de la cartera y rango actual. La barra de progreso hacia el siguiente rango calcula el beneficio de trading descontando el saldo bonus, garantizando que las recompensas de los retos no influyan en el avance del rango. Como se observa en la Figura 5, la lista de activos en cartera muestra el precio actual, la cantidad de unidades, el valor invertido y el beneficio o pérdida de cada posición.

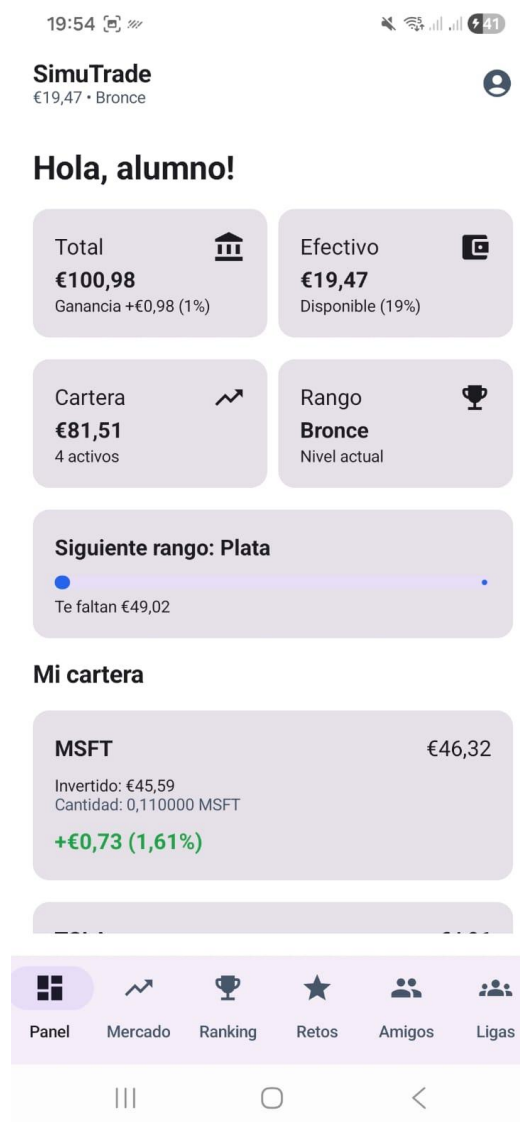


Figura 5. Panel de control con tarjetas de resumen, barra de progreso de rango y lista de cartera. Fuente: captura propia de la aplicación.

4.3.3 Mercado

La pantalla de mercado obtiene los precios de acciones y criptomonedas en tiempo real mediante las APIs de Finnhub (2024) y CoinGecko (2024) respectivamente, utilizando Retrofit (Square, 2024) para las peticiones HTTP. Para respetar el límite de peticiones de CoinGecko en el plan gratuito, se ha implementado un sistema de cooldown de 60 segundos entre actualizaciones manuales y un autorefresh automático cada 2 minutos. En caso de recibir un error 429 (demasiadas peticiones), los datos anteriores se mantienen en pantalla en lugar de mostrar un estado de error.

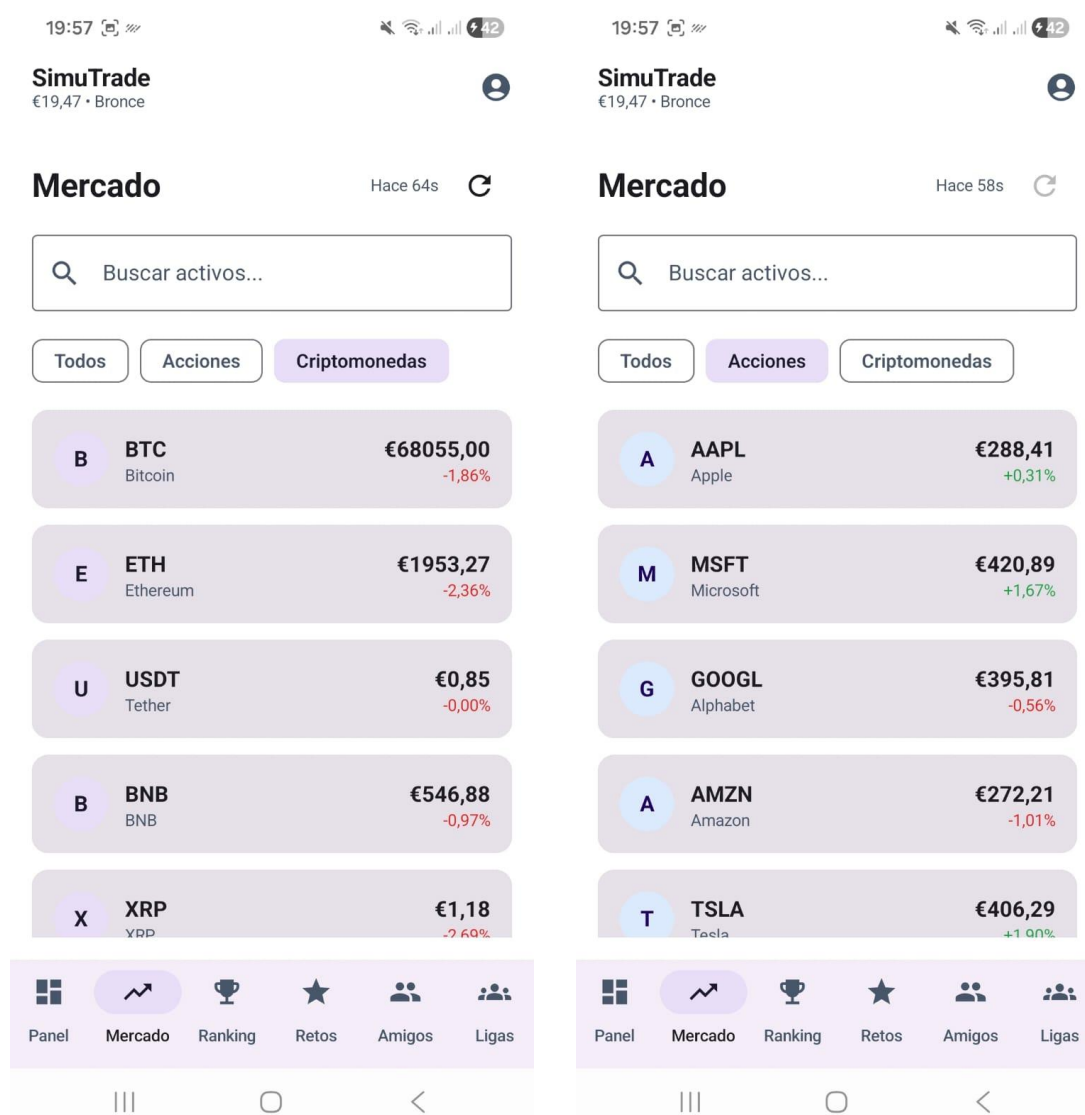


Figura 6. Pantalla de mercado con lista de acciones y criptomonedas, precios reales y filtros.
Fuente: captura propia de la aplicación.

4.3.4 Operaciones de Trading

La pantalla de operaciones permite al usuario comprar o vender el activo previamente seleccionado en la pantalla de mercado. Se muestran el precio actual, el saldo disponible y, si el usuario ya posee el activo, la posición actual con el precio promedio de compra. El UsuarioViewModel utiliza un Mutex de Kotlin para prevenir operaciones simultáneas que pudieran causar inconsistencias en el saldo.



Figura 7. Pantalla de operaciones con compra y venta, validaciones de saldo y cantidad.
Fuente: captura propia de la aplicación.

4.3.5 Ranking Global

El leaderboard global muestra a todos los usuarios ordenados por su beneficio de trading, que se calcula y almacena en el campo beneficio de Firestore cada vez que el usuario realiza una operación. La tarjeta superior destaca la posición propia del usuario mostrando su beneficio, valor total, efectivo y rango actual. Los beneficios positivos se muestran en verde, los negativos en rojo y el valor cero en color neutro, como se aprecia en la Figura 8.

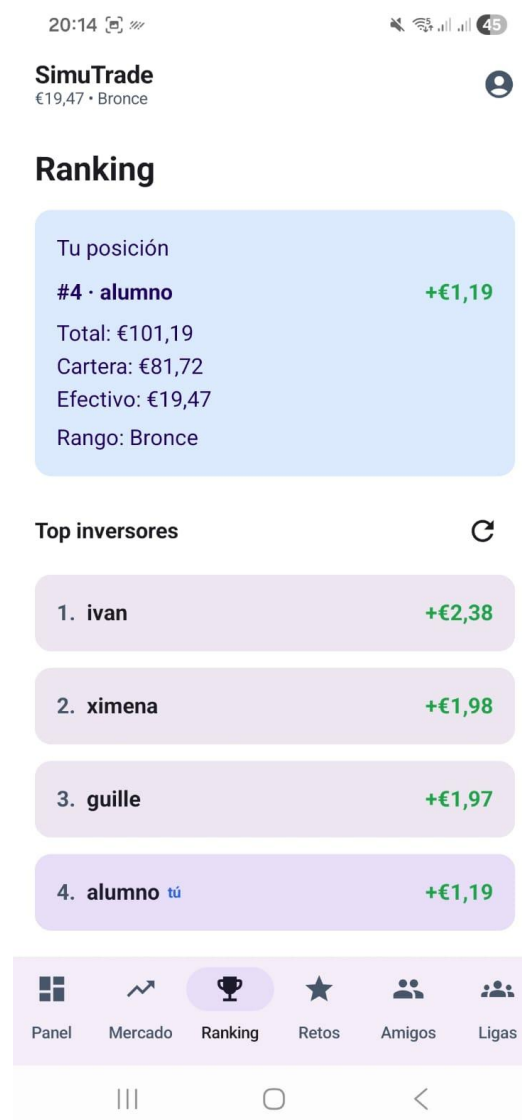


Figura 8. Ranking global con tarjeta de posición propia y lista de inversores ordenados por beneficio. Fuente: captura propia de la aplicación.

4.3.6 Retos Diarios

El sistema de retos genera 3 retos aleatorios al inicio de cada día a partir de un pool de 5 tipos: realizar una operación, diversificar la cartera, realizar 3 operaciones, tener activos de ambos tipos (acciones y criptos) y estar en beneficio. El cálculo de beneficio para este último reto descuenta el saldo bonus, manteniendo la coherencia con el resto del sistema. Al pulsar el botón Completar, el RetosViewModel valida en tiempo real contra Firestore si se cumplen las condiciones. Si se completan todos los retos del día, la racha se incrementa inmediatamente en la base de datos y se muestra un contador regresivo hasta la medianoche.



Figura 9. Pantalla de retos diarios con racha, barra de progreso y ejemplo de reto completado.
Fuente: captura propia de la aplicación.

4.3.7 Sistema de Amigos

El sistema social permite buscar usuarios mediante su código identificativo #XXXXXXXX. Al encontrar un usuario, se puede enviar una solicitud de amistad que aparece en la pantalla de amigos del destinatario. Cuando el destinatario acepta la solicitud, se ejecuta una operación en Firestore que añade la relación de amistad en ambos usuarios y elimina la solicitud de forma atómica, garantizando la consistencia de los datos.

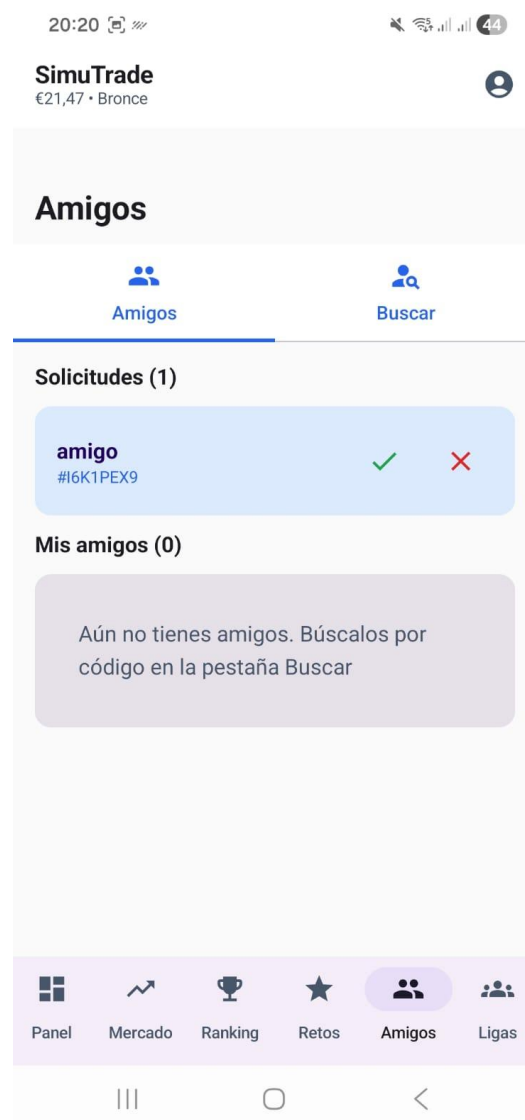


Figura 10. Pestaña Amigos con lista de amigos actuales y solicitudes pendientes. Fuente: captura propia de la aplicación.



Figura 11. Pestaña Buscar con campo de búsqueda por código y resultado con botón de agregar. Fuente: captura propia de la aplicación.

4.3.8 Sistema de Ligas y Chat

Las ligas permiten crear grupos privados de competición con nombre personalizado. Cualquier miembro de una liga puede invitar a sus amigos, y los usuarios invitados reciben la invitación en su pantalla de Ligas. Cada liga tiene su propio ranking calculado con los datos reales de Firestore de cada miembro. Los usuarios pueden salir de cualquier liga en cualquier momento sin necesidad de autorización del creador. Si el último miembro abandona la liga, esta se elimina automáticamente de Firestore.

Además del ranking interno, cada liga incorpora un chat en tiempo real que permite a los miembros comunicarse entre sí. El chat utiliza el mecanismo `addSnapshotListener` de Firestore, que actualiza la lista de mensajes de forma instantánea sin necesidad de recargar la pantalla. El historial completo de mensajes se almacena de forma permanente en la subcolección mensajes de cada liga. Las burbujas propias se muestran a la derecha en color azul y las ajenas a la izquierda, con el nombre del remitente visible encima de cada mensaje.

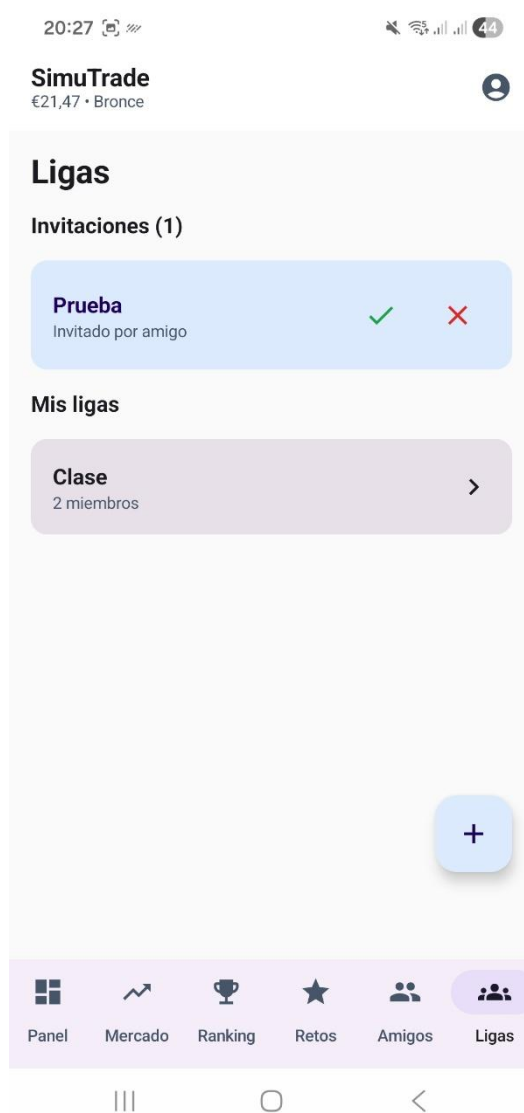


Figura 12. Pantalla de ligas con lista de ligas activas e invitaciones pendientes. Fuente: captura propia de la aplicación.



Figura 13. Pantalla de detalle de liga con tabs de Ranking y Chat en tiempo real. Fuente: captura propia de la aplicación.

4.3.9 Perfil de Usuario

El perfil es accesible desde el icono de la barra superior. Muestra el nombre de usuario, el correo electrónico, el código identificativo #XXXXXXXXX en color azul para facilitar su identificación, el saldo disponible, el saldo inicial de referencia, el bonus acumulado por retos completados y el rango actual con su color e icono correspondiente. Se incluye un diálogo de confirmación antes de cerrar sesión para evitar cierres accidentales.

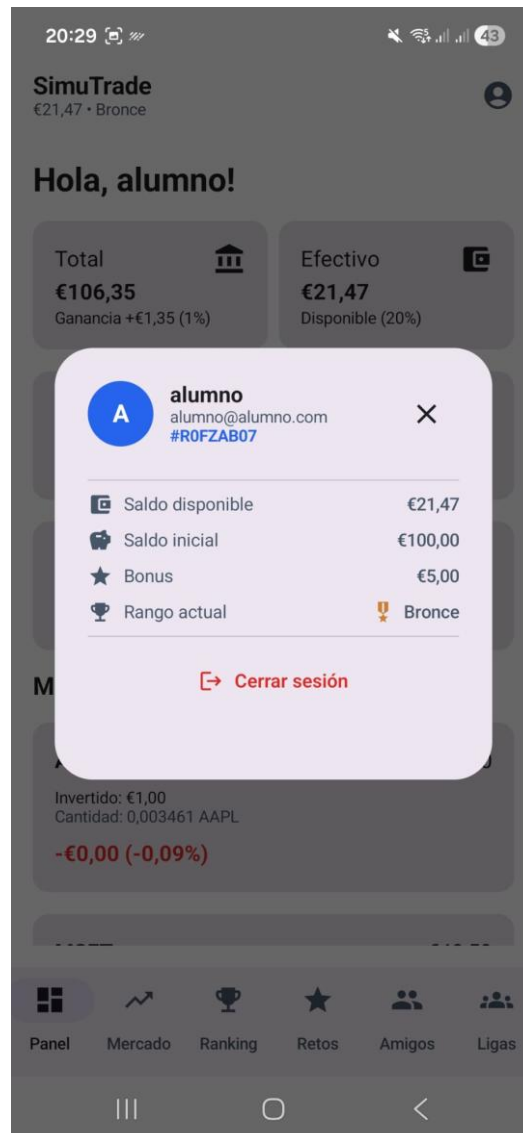


Figura 14. Diálogo de perfil de usuario con datos personales, código identificativo y rango actual. Fuente: captura propia de la aplicación.

5. Pruebas Realizadas y Validación

Se han realizado pruebas manuales sobre un emulador Android API 36 y sobre un dispositivo físico, verificando tanto el comportamiento esperado en el flujo normal como los casos de error y los casos límite.

5.1 Pruebas de Autenticación

Tabla 10. Resultados de las pruebas — registro e inicio de sesión.

Caso de prueba	Resultado esperado	Resultado
Registro con datos válidos	Usuario creado, código generado, navega al Dashboard.	Correcto
Registro con email ya existente	Mensaje de error: el correo ya está registrado.	Correcto
Inicio de sesión con credenciales incorrectas	Mensaje de error: credenciales no válidas.	Correcto
Inicio de sesión con Google	Autenticación correcta, acceso a la aplicación.	Correcto
Campo saldo_bonus inicializado	Valor 0.0 en el documento de Firestore.	Correcto

Nota. Fuente: elaboración propia.

```
beneficio: 1.35
codigo_usuario: "ROFZAB07"
creado_en: 1777824476692
email: "alumno@alumno.com"
mis_ligas
  0 "RxVu6PQTY1LRR2XgfgHS"
nombre_usuario: "alumno"
saldo: 21.47
saldo_bonus: 5
saldo_inicial: 100
ultimo_login: 1778176272241
valor_cartera: 84.88
```

Figura 15. Documento del usuario en Firebase Console tras el registro mostrando todos los campos. Fuente: captura propia de la aplicación.

5.2 Pruebas de Trading

Tabla 11. Resultados de las pruebas — compra y venta de activos.

Caso de prueba	Resultado esperado	Resultado
Compra con saldo suficiente	Saldo reducido, activo en cartera, transacción guardada.	Correcto
Compra con saldo insuficiente	Mensaje de error, sin cambios en saldo ni cartera.	Correcto
Venta de cantidad exacta disponible	Activo eliminado de la cartera, saldo incrementado.	Correcto
Venta de cantidad superior a disponible	Mensaje de error, sin cambios.	Correcto
Beneficio de trading no incluye bonus	La barra de progreso de rango no avanza con recompensas de retos.	Correcto

Nota. Fuente: elaboración propia.

5.3 Pruebas del Sistema de Retos

Tabla 12. Resultados de las pruebas — sistema de retos.

Caso de prueba	Resultado esperado	Resultado
Reclamar reto sin condiciones cumplidas	Mensaje indicando qué falta por cumplir.	Correcto
Reclamar reto con condiciones cumplidas	Saldo bonus aumenta, reto marcado como completado.	Correcto
Completar todos los retos del día	Racha sube en Firestore, contador regresivo visible.	Correcto
Reset al día siguiente	Nuevos retos generados, lista de completados reseteada.	Correcto
Racha rota por día sin completar todos	Racha vuelve a cero al día siguiente.	Correcto
Reto beneficio descuento saldo bonus	Solo el beneficio de trading real determina si el reto se cumple.	Correcto

Nota. Fuente: elaboración propia.

5.4 Pruebas del Sistema Social y Chat

Tabla 13. Resultados de las pruebas — sistema social y chat.

Caso de prueba	Resultado esperado	Resultado
Buscar código válido existente	Se muestra el usuario encontrado con botón Agregar.	Correcto
Buscar código inexistente	Mensaje: no se ha encontrado ningún usuario.	Correcto
Enviar solicitud de amistad	La solicitud aparece en la pantalla del destinatario.	Correcto
Aceptar solicitud de amistad	Ambos usuarios se añaden mutuamente como amigos.	Correcto
Crear liga con nombre válido	Liga creada en Firestore, creador como miembro aceptado.	Correcto
Invitar amigo a liga	Amigo recibe invitación en su pantalla de Ligas.	Correcto
Aceptar invitación a liga	Usuario aparece en el ranking de la liga.	Correcto
Salir de liga siendo el único miembro	La liga se elimina completamente de Firestore.	Correcto
Enviar mensaje en el chat de una liga	El mensaje aparece en tiempo real en el chat de todos los miembros.	Correcto
Enviar mensaje vacío	El sistema impide el envío, el botón permanece desactivado.	Correcto
Volver a entrar al chat de una liga	Se muestran todos los mensajes anteriores cargados desde Firestore.	Correcto

Nota. Fuente: elaboración propia.

6. Planificación del Proyecto

El desarrollo de SimuTrade se ha llevado a cabo de forma iterativa a lo largo del curso académico 2025/2026. La Tabla 14 resume las fases del proyecto con sus periodos de ejecución y el estado final de cada una.

Tabla 14. Planificación del proyecto por fases.

Fase	Actividad	Periodo	Estado
1	Análisis, diseño y arquitectura del sistema	Oct – Dic 2025	Completado
2	Autenticación con Firebase y navegación base	Ene 2026	Completado
3	Integración de APIs de mercado y trading	Feb 2026	Completado
4	Sistema de rangos, retos y gamificación	Mar 2026	Completado
5	Sistema social: amigos, ligas y chat	Abr 2026	Completado
6	Pruebas, ajustes finales y documentación	May 2026	Completado

Nota. Fuente: elaboración propia.

7. Conclusiones y Mejoras Futuras

7.1 Objetivos Alcanzados

Todos los objetivos propuestos al inicio del proyecto han sido alcanzados satisfactoriamente. Se ha desarrollado una aplicación Android funcional y completa que supera en funcionalidades el alcance inicial planificado en la Entrega Parcial 1, que preveía únicamente las pantallas básicas de Dashboard, Mercado y Rankings con datos simulados y persistencia local.

Los logros más destacados del proyecto son los siguientes:

- Se ha implementado una arquitectura MVVM limpia y escalable con repositorios separados por dominio funcional, lo que facilita el mantenimiento y la extensión futura del código.
- Se ha integrado un backend completo con Firebase (2024) sin necesidad de servidor propio, garantizando persistencia, sincronización en tiempo real y autenticación segura.
- Se ha resuelto el problema de equidad del ranking garantizando que las recompensas de los retos diarios no influyan en el beneficio de trading ni en el leaderboard.
- Se ha implementado un sistema social completo con amigos, ligas privadas y chat en tiempo real, superando con creces el alcance de proyectos similares de ciclo.

7.2 Dificultades Encontradas

La principal dificultad técnica encontrada durante el desarrollo ha sido garantizar la consistencia de los datos en Firestore cuando múltiples operaciones deben ejecutarse de forma atómica. Esta problemática se ha resuelto mediante el uso de batch writes de Firestore para operaciones que afectan a varios documentos relacionados simultáneamente, especialmente en el sistema social de amigos y ligas, y mediante el uso de un Mutex de Kotlin en el UsuarioViewModel para prevenir operaciones de trading simultáneas.

Otra dificultad significativa ha sido la gestión del rate limiting de la API gratuita de CoinGecko (2024), que limita el número de peticiones por minuto. Se ha implementado un sistema de cooldown y caché local que mantiene los datos anteriores en pantalla cuando se produce un error 429, evitando mostrar estados de error al usuario.

7.3 Posibles Mejoras Futuras

Se han identificado las siguientes vías de mejora y ampliación para versiones posteriores de la aplicación:

- Implementación de foto de perfil de usuario almacenada en Firebase Storage, con selector de imagen desde la galería del dispositivo. Esta funcionalidad fue considerada durante el desarrollo, pero no pudo completarse debido a restricciones en la configuración del bucket de Firebase Storage en el plan gratuito.
- Gráficos de evolución de precios de cada activo con selector de periodo mediante la librería Vico o MPAndroidChart.
- Notificaciones push mediante Firebase Cloud Messaging para alertar al usuario cuando reciba una solicitud de amistad, una invitación a una liga o un nuevo mensaje en el chat.
- Modo educativo con preguntas tipo test sobre conceptos financieros básicos y recompensas en saldo bonus al completarlas.
- Ampliación del catálogo de activos con ETFs, fondos indexados y materias primas como el oro o el petróleo.
- Panel de administración web para gestionar usuarios, activos disponibles y configurar los parámetros de los retos diarios.

Bibliografía

Android Developers. (2024a). Guide to app architecture. Google.

<https://developer.android.com/topic/architecture>

Android Developers. (2024b). Jetpack Compose. Google.

<https://developer.android.com/jetpack/compose>

Android Developers. (2024c). Kotlin coroutines on Android. Google.

<https://developer.android.com/kotlin/coroutines>

CoinGecko. (2024). CoinGecko API v3 documentation.

<https://docs.coingecko.com>

Firebase. (2024a). Cloud Firestore documentation. Google.

<https://firebase.google.com/docs/firestore>

Firebase. (2024b). Firebase Authentication documentation. Google.

<https://firebase.google.com/docs/auth>

Finnhub. (2024). Finnhub Stock API documentation. <https://finnhub.io/docs/api>

Google. (2024). Material Design 3. <https://m3.material.io>

JetBrains. (2024). Kotlin documentation. <https://kotlinlang.org/docs/home.html>

Square. (2024). Retrofit: A type-safe HTTP client for Android and Java.

<https://square.github.io/retrofit>

Anexos

Anexo I — Manual de Usuario

A.1.1 Registro e inicio de sesión

Para utilizar SimuTrade es necesario crear una cuenta. Es posible registrarse introduciendo un nombre de usuario, una dirección de correo electrónico y una contraseña de al menos 6 caracteres, o bien utilizando una cuenta de Google pulsando el botón Continuar con Google. Una vez registrado, el sistema asigna automáticamente un código identificativo de 8 caracteres con el formato #XXXXXXXX que puede consultarse en el perfil de usuario.

A.1.2 Operar en el mercado

Para comprar un activo, se debe acceder a la pestaña Mercado, pulsar sobre el activo deseado e introducir la cantidad que se desea adquirir. El sistema verificará que el saldo disponible es suficiente antes de ejecutar la operación. Para vender, se puede acceder al activo desde la cartera en el Panel de Control o desde la pantalla de Mercado si ya se posee.

A.1.3 Sistema de retos

Cada día se generan 3 retos nuevos que se resetean a medianoche. Para reclamar la recompensa de un reto, es necesario cumplir las condiciones indicadas (por ejemplo, realizar una operación de compra o venta durante el día) y pulsar el botón Completar. Si las condiciones no se cumplen, el sistema indica exactamente qué falta por hacer. La recompensa se añade al saldo bonus, que no afecta al rango ni al leaderboard.

A.1.4 Añadir amigos y crear ligas

Para añadir un amigo, se debe acceder a la pestaña Amigos, seleccionar la pestaña Buscar e introducir el código #XXXXXXXX del usuario que se desea agregar. Una vez aceptada la solicitud, ambos usuarios pueden verse mutuamente en su lista de amigos y pueden invitarse a ligas. Para crear una liga, se accede a la pestaña Ligas y se pulsa el botón +. Una vez dentro de una liga,

es posible acceder al chat pulsando la pestaña Chat en la pantalla de detalle de la liga.

Anexo II — Manual de Instalación y Ejecución

A.2.1 Requisitos previos

Para compilar y ejecutar SimuTrade es necesario disponer de Android Studio (versión Meerkat 2024.3.1 o superior), JDK 17 o superior, conexión a Internet y un dispositivo Android con API mínima 24 (Android 7.0) o un emulador configurado en Android Studio.

A.2.2 Clonado e importación del proyecto

El proyecto debe clonarse desde el repositorio de GitHub mediante el comando `git clone` y abrirse en Android Studio con la opción `File → Open`. A continuación, se debe esperar a que Gradle sincronice todas las dependencias automáticamente.

A.2.3 Configuración de Firebase

La aplicación requiere un proyecto de Firebase con Firebase Authentication y Cloud Firestore activados. El archivo `google-services.json` descargado desde Firebase Console debe colocarse en el directorio `app/` del proyecto. Para Google Sign-In, es necesario registrar la huella digital SHA-1 del certificado de debug en Firebase Console.

A.2.4 Configuración de la API de Finnhub

La clave de API de Finnhub debe añadirse en el archivo `local.properties` en la raíz del proyecto con el formato `FINNHUB_API_KEY=tu_clave_aqui`. Este archivo no se sube al repositorio de control de versiones por motivos de seguridad. Es posible obtener una clave gratuita en <https://finnhub.io/register>.

A.2.5 Ejecución

Una vez sincronizado el proyecto y configurados los servicios externos, se selecciona un emulador o dispositivo físico en Android Studio y se pulsa el botón Run. La primera compilación puede tardar varios minutos mientras se descargan todas las dependencias.

A.2.6 Instalación mediante APK

También es posible instalar la aplicación mediante el archivo APK generado con Build → Build APK(s). En el dispositivo destino debe activarse previamente la instalación desde orígenes desconocidos en los ajustes de seguridad del sistema.

Anexo III — Declaración de Originalidad

Los autores del presente proyecto, Guillermo Sierra Castejón, Iván Bertolo García y Ximena Meyzen Calderón, declaran que el trabajo presentado es original y ha sido elaborado íntegramente por el equipo durante el curso académico 2025/2026, a excepción de las fuentes y recursos externos debidamente citados en la bibliografía.

Se declara asimismo que no se ha utilizado código de terceros sin la correspondiente atribución, y que el uso de herramientas de inteligencia artificial ha sido supervisado y verificado por los autores en todo momento.